

C++ Programming Textbook Notes

Module: Constructors in C++ | Types, Compiler Rules & Memory Initialization

Author: Gaurav Kandpal

Platform: CS-Simplified

Level: Beginner to Advanced

1. CONSTRUCTORS IN C++: CONCEPTUAL FOUNDATIONS

What is a Constructor?

A **Constructor** is a special member function of a class whose primary task is to initialize the data members of an object when it is instantiated. It guarantees that every object begins its lifetime in a valid, well-defined initial state.

Why Do We Need Constructors?

In C++, declaring an object without initialization leaves its primitive member variables populated with **garbage values** (indeterminate bit patterns in RAM). Calling separate setter functions (such as `c1.setData(3, 4);`) relies on manual programmer discipline; if forgotten, subsequent calculations produce corrupted outputs. Constructors solve this by binding initialization directly to memory allocation.

Core Characteristics of Constructors

- **Identical Name:** The constructor's name must be **identical to the class name**.
- **No Return Type:** It has **no return type** (not even `void`), and therefore cannot use the `return` keyword to return values.
- **Automatic Invocation:** A constructor is not called explicitly like ordinary functions; it is **implicitly invoked** by the compiler at the precise instant an object is created.
- **Cannot Be Static:** Constructors can **never be static** because they must operate on an individual object instance bound via the `this` pointer.
- **Must Reside in Public:** Usually placed in the `public` section so external functions (like `main()`) can instantiate objects.

How Constructors Work in Memory

When an object statement like `Complex c1;` executes:

1. The operating system reserves stack memory (e.g., 8 bytes for two integers `a` and `b`).
2. The compiler immediately passes the newly allocated memory address (via the implicit `this` pointer) to the constructor routine.
3. The constructor executes its instructions, assigning valid initial values to `a` and `b` before control moves to the next program statement.

[CONSTRUCTOR EXECUTION & RAM INITIALIZATION FLOW]

Source Code Statement: `Complex c1(3, 4);`

Step 1: RAM Block Reserved on Stack (8 Bytes at 0x7ffd10)

int a: [Garbage Bits]	int b: [Garbage Bits]
-------------------------	-------------------------



Step 2: Implicit Constructor Invocation -> `Complex::Complex(int x, int y)`
Passes 0x7ffd10 as 'this', x = 3, y = 4



Step 3: Initialized Object Memory (Guaranteed Valid State)

int a = 3	int b = 4
-----------	-----------

Address: 0x7ffd10

Address: 0x7ffd14

2. TYPES OF CONSTRUCTORS IN C++

C++ supports three primary types of constructors based on parameter signatures:

1. Default Constructor (Zero Arguments)

A constructor that accepts **no arguments** (or where all arguments have default values). It initializes objects when instantiated without any argument list: `Complex c1;`.

2. Parameterized Constructor (With Arguments)

A constructor that accepts one or more arguments to initialize an object's state with customized values at the time of creation: `Complex c1(3, 4);`.

3. Copy Constructor (Object-to-Object Initialization)

A constructor that initializes a new object by copying data members from an already existing object of the same class: `Complex c4(c1);` or `Complex c4 = c1;`.

Signature: `ClassName(const ClassName &source);` (Must pass by reference!).

3. COMPILER DEFAULT CONSTRUCTOR GENERATION RULES

Understanding how and when the C++ compiler automatically generates constructors is essential to prevent confusing compilation errors:

[COMPILER CONSTRUCTOR SYNTHESIS RULES SUMMARY]

Condition in User Code:

1. Zero constructors defined by user.
2. User defines a Default Constructor.
3. User defines a Parameterized Constructor.
4. User defines a Copy Constructor.

Compiler Action:

- Compiler creates: Default + Copy Constructor.
- Compiler creates: Copy Constructor only.
- Compiler creates: Copy Constructor only.
(NO Default Constructor is generated!)
- Compiler creates: NOTHING.

The Missing Default Constructor Trap

If you define a parameterized constructor (e.g., `Complex(int x, int y)`), the compiler **revokes its automatic default constructor**! If you subsequently attempt to write `Complex c2;` without arguments, the compiler flags a fatal error: *"no matching default constructor available"*. Always define an explicit default constructor if parameterless instantiation is needed!

4. SYNTAX & INVOCATION VARIETIES

Constructor Variety	Implicit Notation (Functional)	Explicit Notation (Assignment)
Default Constructor	<code>Complex c1;</code> (Do not write <code>Complex c1();</code>)	<code>Complex c1 = Complex();</code>
Parameterized (2 Args)	<code>Complex c1(3, 4);</code>	<code>Complex c1 = Complex(3, 4);</code>
Single-Argument Conversion	<code>Complex c2(5);</code>	<code>Complex c2 = 5;</code> (Implicit conversion)
Copy Constructor	<code>Complex c4(c1);</code>	<code>Complex c4 = c1;</code>

Most Vexing Parse Warning

Writing `Complex c1();` does **not** call the default constructor. The C++ compiler treats it as a *function declaration* named `c1` that takes zero parameters and returns a `Complex` object! Always declare parameterless objects as `Complex c1;`.

5. PRACTICAL CODE DEMONSTRATIONS

Example 1: Default & Parameterized Constructors

```
// Demonstrating Default and Parameterized Constructors in C++
#include <iostream>
using namespace std;

class Complex {
private:
    int a, b;

public:
    // 1. Default Constructor (Zero arguments)
    Complex() {
        a = 0;
        b = 0;
        cout << "Default constructor invoked!" << endl;
    }

    // 2. Parameterized Constructor with 2 arguments
    Complex(int x, int y) {
        a = x;
        b = y;
        cout << "Parameterized constructor (2 args) invoked!" << endl;
    }

    // 3. Parameterized Constructor with 1 argument (Single-arg conversion)
    Complex(int k) {
        a = k;
        b = 0;
        cout << "Parameterized constructor (1 arg) invoked!" << endl;
    }

    void showData() {
        cout << "a = " << a << " | b = " << b << endl;
    }
};

int main() {
    // Calls Default Constructor
    Complex c1;
    c1.showData();

    // Calls Parameterized Constructor (2 args)
    Complex c2(3, 4);
    c2.showData();

    // Calls Parameterized Constructor (1 arg via assignment syntax)
    Complex c3 = 5; // Equivalent to Complex c3(5);
    c3.showData();

    return 0;
}
```

Example 2: Copy Constructor Implementation

```
// Complete Demonstration of Copy Constructor
#include <iostream>
using namespace std;

class Complex {
private:
    int a, b;

public:
    // Parameterized Constructor
    Complex(int x, int y) {
        a = x;
        b = y;
    }

    // Explicit Copy Constructor (Must pass by reference using &)
    Complex(const Complex &c) {
        a = c.a;
        b = c.b;
        cout << "Copy constructor invoked successfully!" << endl;
    }

    void showData() {
        cout << "a = " << a << " | b = " << b << endl;
    }
};

int main() {
    Complex c1(10, 20); // Calls Parameterized Constructor

    // Invoking Copy Constructor: Notation 1
    Complex c2(c1);
    c2.showData();

    // Invoking Copy Constructor: Notation 2
    Complex c3 = c1;
    c3.showData();

    return 0;
}
```

Why Pass-By-Reference is Mandatory in Copy Constructors

In a copy constructor declaration `Complex(Complex c)` without `&`, passing `c` by value requires creating a copy of `c`. Creating a copy of `c` calls the copy constructor again, which calls it again, resulting in **infinite recursive calls** that crash the stack! Passing by reference (`Complex(const Complex &c)`) resolves this cleanly.